

CRUD-BASED WEB APPLICATION

Project Report

Name: Neka SK

Register Number: 922525104101

Department: Computer Science and Engineering

Academic Year: 2026–2027

1. ABSTRACT

This project is a CRUD-based web application developed to manage [students/employees/books/inventory/etc.] in a simple and organized way.

CRUD stands for Create, Read, Update, and Delete. These four operations are the basic functions used to manage data in many real-world applications.

The application allows users to add new records, view existing records, update information when required, and delete records that are no longer needed. The frontend provides an easy-to-use interface, while the backend handles the application logic and communicates with the database through REST APIs.

The project also includes input validation, error handling, API testing, and database integration. The main purpose of this project is to understand how the frontend, backend, API, and database work together to form a complete web application.

2. INTRODUCTION

In many organizations, data is still managed using spreadsheets or manual records. This can make it difficult to search, update, and maintain information.

To solve this problem, this project provides a web-based application for managing [your records].

The application provides a simple interface where users can perform different operations on the stored data. Whenever a user adds, edits, views, or deletes information, the request is sent to the backend through a REST API. The backend processes the request and communicates with the database.

This project helped us understand the complete flow of a full-stack web application.

3. PROBLEM STATEMENT

Managing [students/employees/books/inventory/etc.] manually can be time-consuming and may lead to duplicate, missing, or outdated information.

The main problem is to develop a simple web application that can store and manage records efficiently.

The system should allow the user to:

- Add new records
- View existing records
- Edit existing records
- Delete records
- Validate user input
- Search or filter records
- Store data permanently in a database

4. OBJECTIVES

The main objectives of this project are:

1. To develop a simple and user-friendly web application.
2. To implement Create, Read, Update, and Delete operations.
3. To connect the frontend with the backend using REST APIs.
4. To store application data in a database.
5. To implement client-side and server-side validation.
6. To handle invalid inputs and errors properly.
7. To test the REST API using Postman.
8. To understand the basic architecture of a full-stack application.
9. To use Git/GitHub for managing the project source code.

5. TECHNOLOGY STACK

The following technologies were used in this project:

Technology| Purpose

HTML| Creating the basic webpage structure

CSS| Designing and styling the interface

JavaScript| Adding interaction and frontend functionality

[React, if used]| Building the frontend components

[Django / Spring Boot]| Developing the backend

REST API| Communication between frontend and backend

[SQLite / MySQL / PostgreSQL]| Storing application data

Postman| Testing API endpoints

Git| Version control

GitHub| Storing and managing source code

Note: Remove technologies that you did not actually use.

6. SYSTEM ARCHITECTURE

The application follows a basic three-layer architecture.

The user interacts with the frontend through the browser. The frontend sends requests to the backend using REST APIs. The backend processes these requests and communicates with the database.

The basic flow is:

User → Frontend → REST API → Backend → Database

For example, when a user adds a new record, the frontend sends the data to the backend using a POST request. The backend validates the information and stores it in the database. After successful insertion, the result is sent back to the frontend.

Architecture Components

Frontend:

Provides the user interface and allows users to interact with the application.

Backend:

Handles application logic, validation, API requests, and database operations.

REST API:

Acts as a communication layer between the frontend and backend.

Database:

Stores the application records permanently.

7. DATABASE DESIGN

The database is used to store and manage the records used by the application.

The main table used in this project is:

[Table Name]

Field	Data Type	Description
-------	-----------	-------------

ID	Integer	Unique identifier
----	---------	-------------------

Name| String| Name of the record
Email| String| Email address
[Field 1]| [Type]| [Description]
[Field 2]| [Type]| [Description]
[Field 3]| [Type]| [Description]

The ID field is used as the primary key so that every record can be uniquely identified.

Appropriate constraints are used to avoid invalid or duplicate data.

Database Screenshot

Insert your database/table screenshot here.

[INSERT DATABASE SCREENSHOT]

8. FRONTEND DESIGN

The frontend was designed to make the application simple and easy to use.

The main interface contains the required forms and controls for managing records.

The user can:

- Enter information into the form.
- Submit the form to create a record.
- View all available records.
- Select a record for editing.
- Delete an unwanted record.
- Search or filter records if the feature is implemented.

Basic validation is also performed before sending the data to the backend.

9. BACKEND DEVELOPMENT

The backend is responsible for processing requests received from the frontend.

It contains the models/entities, API endpoints, validation logic, and database operations.

The backend receives data from the frontend, checks whether the data is valid, performs the required operation, and sends a response back to the frontend.

The backend also handles errors such as invalid data, missing records, and incorrect record IDs.

10. REST API IMPLEMENTATION

REST APIs are used for communication between the frontend and backend.

The following API operations were implemented:

Operation	HTTP Method	Endpoint	Purpose
Create	POST	"/api/items/"	Add a new record
Read All	GET	"/api/items/"	Get all records
Read One	GET	"/api/items/{id}/"	Get one record
Update	PUT/PATCH	"/api/items/{id}/"	Update a record
Delete	DELETE	"/api/items/{id}/"	Delete a record

POST – Create

The POST request is used to add a new record to the database.

GET – Read

The GET request is used to retrieve existing records from the database.

PUT/PATCH – Update

The PUT or PATCH request is used to modify an existing record.

DELETE – Delete

The DELETE request is used to remove an existing record from the database.

11. CRUD OPERATIONS

11.1 CREATE

The Create operation allows the user to enter new information through the form.

After submitting valid information, the frontend sends the data to the backend. The backend validates the data and stores the new record in the database.

11.2 READ

The Read operation displays the records stored in the database.

The application retrieves the information through a GET API request and displays it on the frontend.

11.3 UPDATE

The Update operation allows users to modify information that has already been stored.

The user selects a record, changes the required information, and saves the changes.

The updated information is then sent to the backend and stored in the database.

11.4 DELETE

The Delete operation allows the user to remove a record from the application.

Before deleting, the application can ask the user for confirmation. After confirmation, a DELETE request is sent to the backend.

12. VALIDATION

Validation is included to make sure that incorrect information is not stored in the database.

The application checks:

- Required fields are not empty.
- Email fields contain a valid email format.
- Numeric fields contain appropriate values.
- Duplicate values are handled properly.
- Invalid data is rejected by the backend.
- Error messages are shown clearly to the user.

Both frontend and backend validation are used because frontend validation alone is not sufficient for protecting the application.

13. SEARCH AND FILTER

A search/filter feature is implemented to help users find records quickly.

Instead of manually checking every record, users can enter a search value and view the matching records.

This makes the application easier to use when the number of records increases.

14. ERROR HANDLING

The application provides appropriate responses when something goes wrong.

For example:

- Required information is missing.
- Invalid data is entered.
- A record with the given ID does not exist.
- Duplicate information is entered.
- The backend is unavailable.
- The database connection fails.

Meaningful error messages are displayed so that users understand what went wrong.

15. FRONTEND-BACKEND INTEGRATION

The frontend communicates with the backend through REST APIs.

When the user performs an operation, the frontend sends an HTTP request to the appropriate API endpoint.

The backend processes the request and returns a response, usually in JSON format.

The frontend then updates the displayed information based on the response.

The overall process is:

User Action → Frontend → API Request → Backend → Database → API Response → Frontend

This integration allows the application to work as a complete full-stack system.

16. API TESTING USING POSTMAN

Postman was used to test the REST API independently from the frontend.

The following operations were tested:

Test	Method	Expected Result	Status
Create valid record	POST	Record created	Passed
Read records	GET	Records displayed	Passed
Read single record	GET	Record displayed	Passed
Update record	PUT/PATCH	Record updated	Passed
Delete record	DELETE	Record deleted	Passed
Missing required field	POST	Validation error	Passed
Invalid ID	GET/PUT/DELETE	Error response	Passed
Duplicate value	POST	Validation error	Passed

21. CHALLENGES FACED

During the development of the project, some challenges were faced.

One challenge was connecting the frontend with the backend API. Understanding how HTTP requests and JSON data move between the two parts of the application required some practice.

Another challenge was handling validation and errors correctly. The application had to make sure that invalid information was not stored in the database.

Database connectivity and CRUD operations also required testing to make sure that changes made through the application were correctly reflected in the database.

These challenges helped us understand how different parts of a full-stack application work together.

22. SOLUTIONS

The problems were solved by testing each part of the application separately.

The API endpoints were tested using Postman before connecting them to the frontend. Input validation was added on both the frontend and backend.

Database operations were tested after every CRUD operation to make sure that the stored information was correct.

Debugging tools and error messages were also used to identify and fix problems during development.

23. FUTURE ENHANCEMENTS

The application can be improved further by adding additional features such as:

- User login and authentication
- Role-based access control
- Advanced search and filtering
- Pagination for large datasets
- Dashboard and analytics
- Exporting data to Excel or PDF
- Email notifications
- Cloud database integration
- Mobile-friendly improvements
- Deployment to a cloud platform

These features can make the application more suitable for real-world usage.

24. CONCLUSION

This project provided practical experience in developing a complete CRUD-based web application.

The application successfully demonstrates Create, Read, Update, and Delete operations along with frontend development, backend APIs, database integration, validation, error handling, and API testing.

Through this project, we gained a better understanding of how different technologies work together in a full-stack application. We also learned the importance of testing, documentation, version control, and proper project structure.

Overall, the project helped us understand the basic workflow of developing and maintaining a real-world web application.